

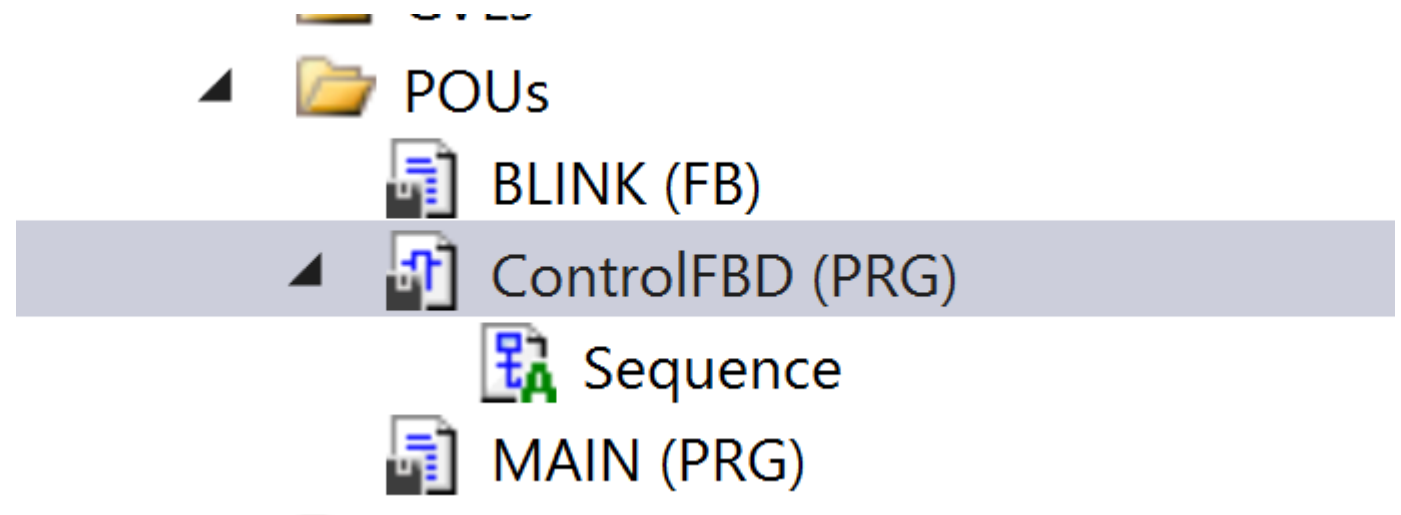
Lecture 5

Programming structure

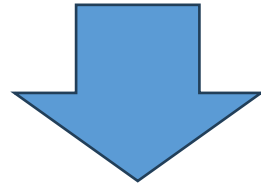
- Programming structure
 - Manual / Auto mode according to the EU Regulation on Machinery
 - Cycle stop
 - Indications
 - Programming Structure
 - Divide in POU's
 - Selection of language
 - Reusable functions and function blocks
 - Data Unit Types DUT (Enum)

Divide in POU

- New PLC project
 - What POU are feasible?
 - Input unit or units
 - Process, Processes
 - Output unit
 - Auto
 - Manual
 - Outputs
- Language
- Global Variable List (GVL)
- Functions
- Function blocks
- Data Unit Types (DUT)



Directive 2006/42/EC of the European Parliament
and of the Council and Council Directive 73/361/EEC
on Machinery



Regulation (EU) 2023/1230 of the European Parliament
and of the Council of 14 June 2023
on Machinery

DIVERSE BETWEEN EMERGENCY STOP & SAFETY STOP

Emergency stop

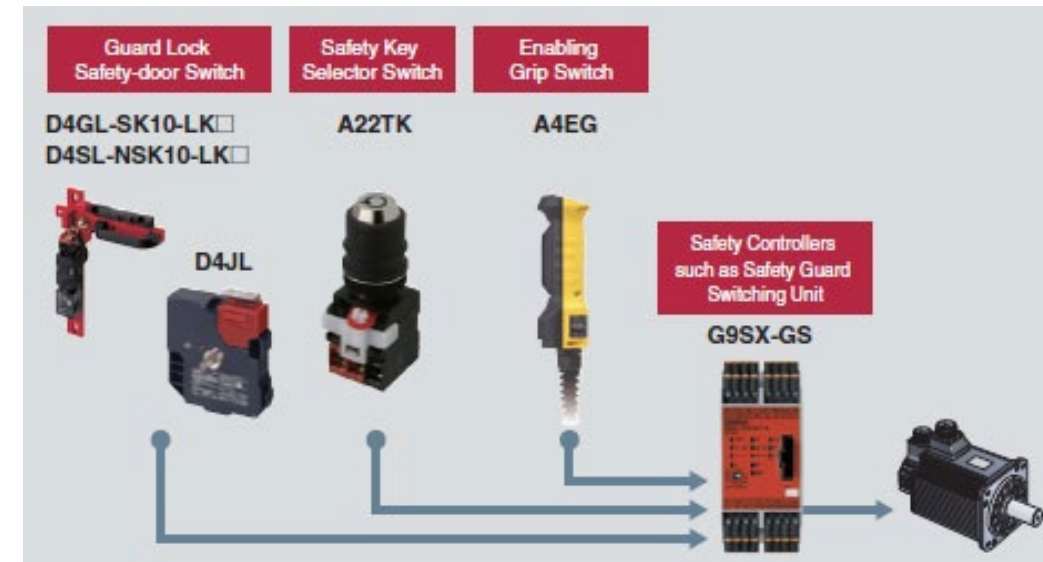
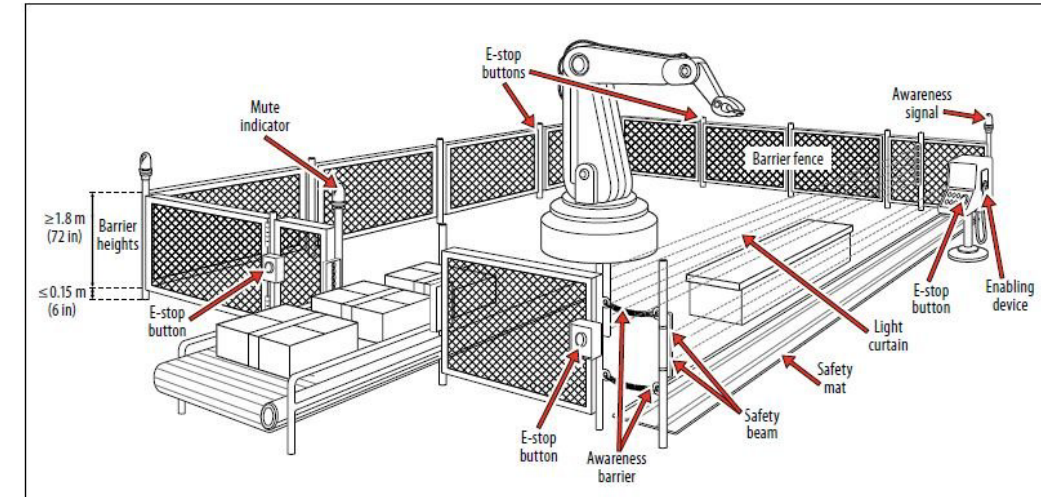
REGULATION (EU) 2023/1230 OF THE
EUROPEAN PARLIAMENT AND OF THE COUNCIL of 14 June 2023 on machinery



- Emergency buttons
 - All buttons must stop all motions and **disconnect all energy**
(Fly of tools, parts, etc. must be secured)
 - All emergency buttons should stop all **visible** machines
 - Emergency button shall **stay in** and must be released manually (Twist, Pull Out, Key)
 - The reset emergency stop button must be in a place that has **free sight** of the cell or zone
 - The automatic motions should **not start directly** after resetting the emergency stop
 - Indications must be **clear and visible**

Safety stop REGULATION (EU) 2023/1230 OF THE EUROPEAN PARLIAMENT AND OF THE COUNCIL of 14 June 2023 on machinery

- Safety stop generates from
 - Doors and Gates in fences around the machine
 - Floor scanners
 - Light beams
 - Light curtains
 - Safety mats
 - Enabling devices
- Stops all auto motions
- Manual motions can be allowed (Risk Assessment)
- All safety must be supervised by a Safety PLC or a Safety Controller



Manual and Auto mode selector

- According to REGULATION (EU) 2023/1230 OF THE EUROPEAN PARLIAMENT AND OF THE COUNCIL of 14 June 2023 on machinery
 - A clearly visible Mode selector must exist on machines that are able to operate in auto mode
 - Clear disguise difference between manual and auto mode



Manual (hand/inch) mode



- Manual motion when all basic condition is fulfilled
 - The mode selector must be in manual mode
 - All motor protections ok
 - All fuses ok
 - All pressures ok (Air, Oil, Hydraulic,...)
 - Emergency stop ok
- Depending on risk assessment it can be ok to run in manual mode with “open gates”
 - Enabling grip switch
 - Reduced speed by 250 mm/s

Service mode

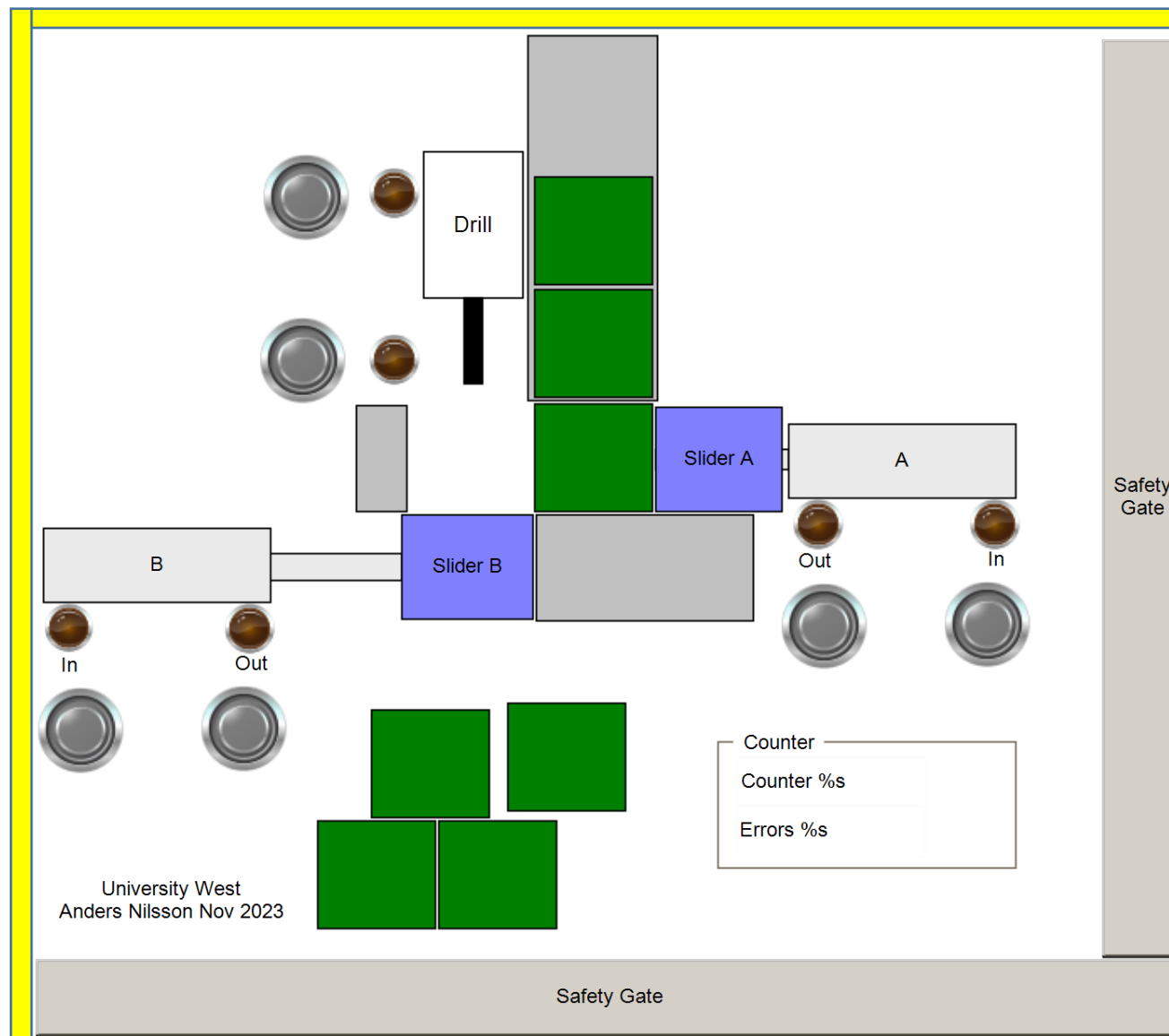


- Manual motion has **interlocks** protect from damage to the machine
- The service mode is for service/maintenance without interlocks
- Often only possible to activate by key or code

Auto mode

- Mode selector in **auto mode**
- **Start button** to start the auto
 - Safety and Em-stop must reset and OK
 - **Home position** for start auto is often needed
- **Cycle stop** for machine-safe and convenient stop and easy start
- The button **start auto** must be from a place that has **free sight** of the machine





Emergency Stop 1

Reset safety

Emergency stop



Safety



Reset safety



Home Pos



Start Auto



Cycle stop



Aut Man

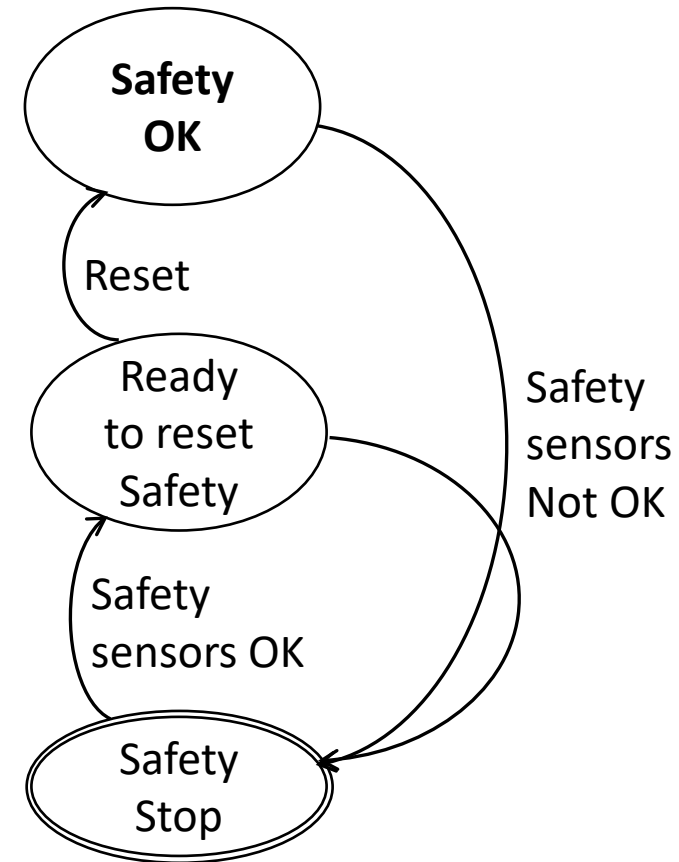
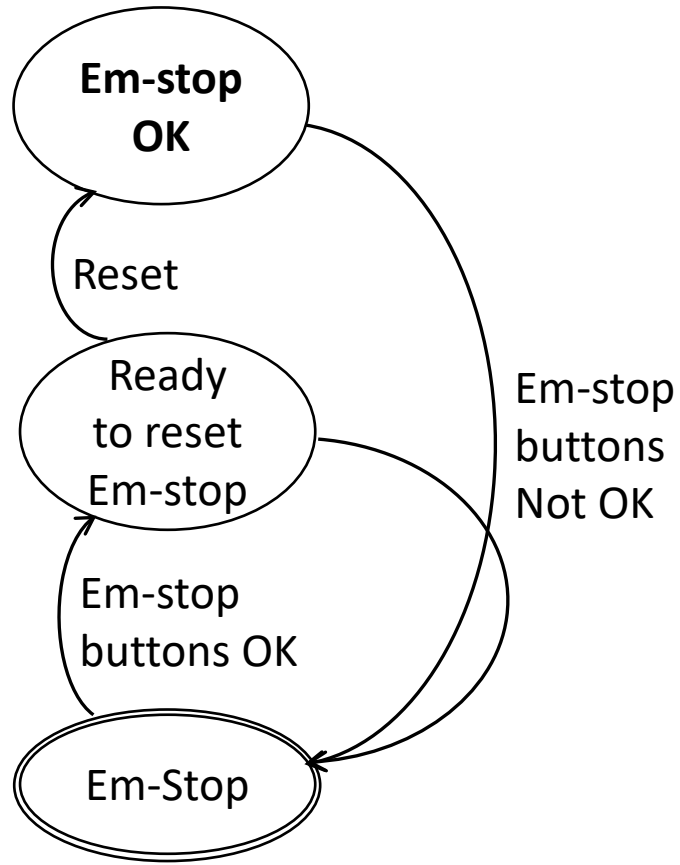
Reset



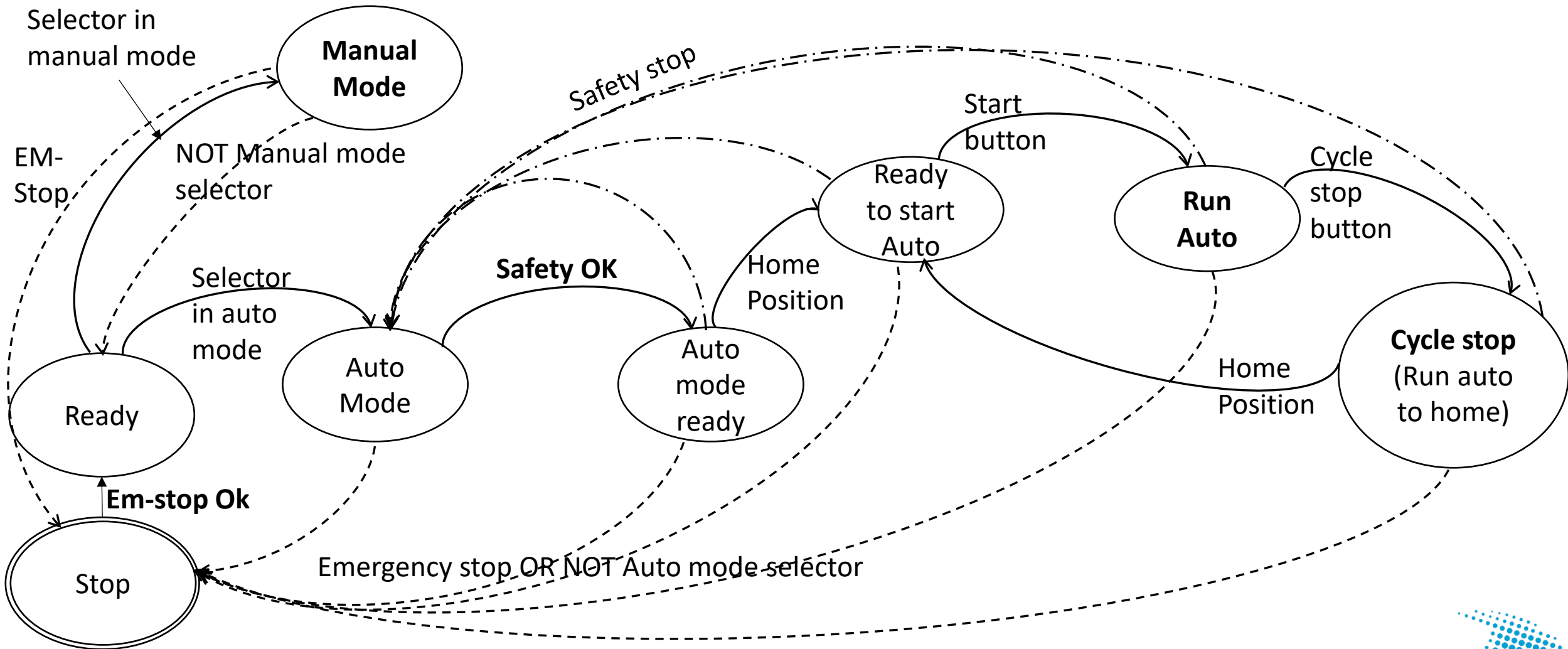
Emergency Stop2



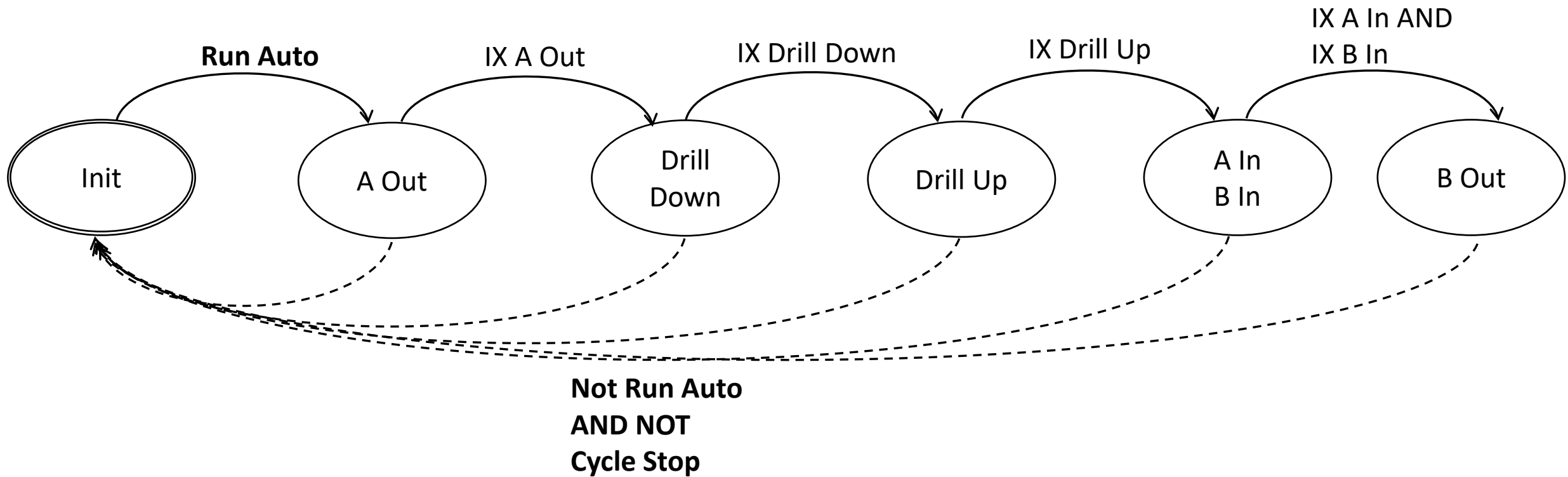
State diagram of Emergency stop (EM-Stop) and Safety stop



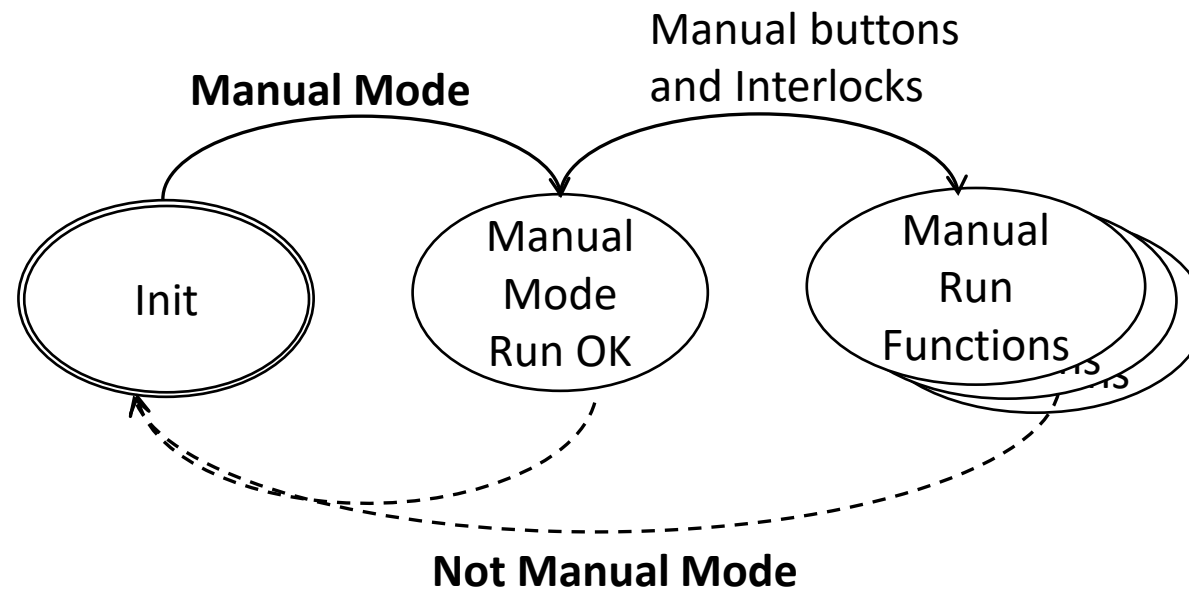
State diagram of manual and auto mode



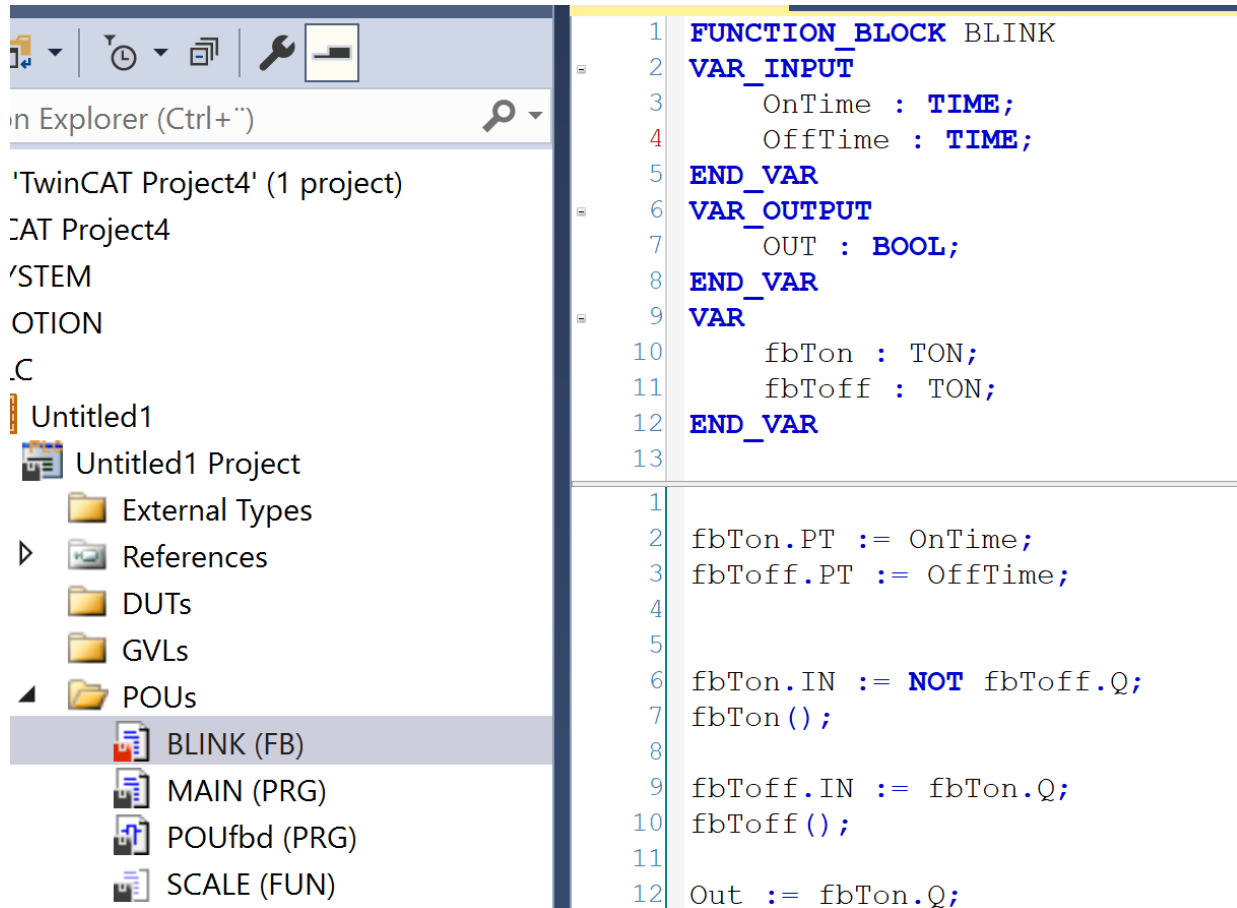
Sequence in auto mode



Manual mode

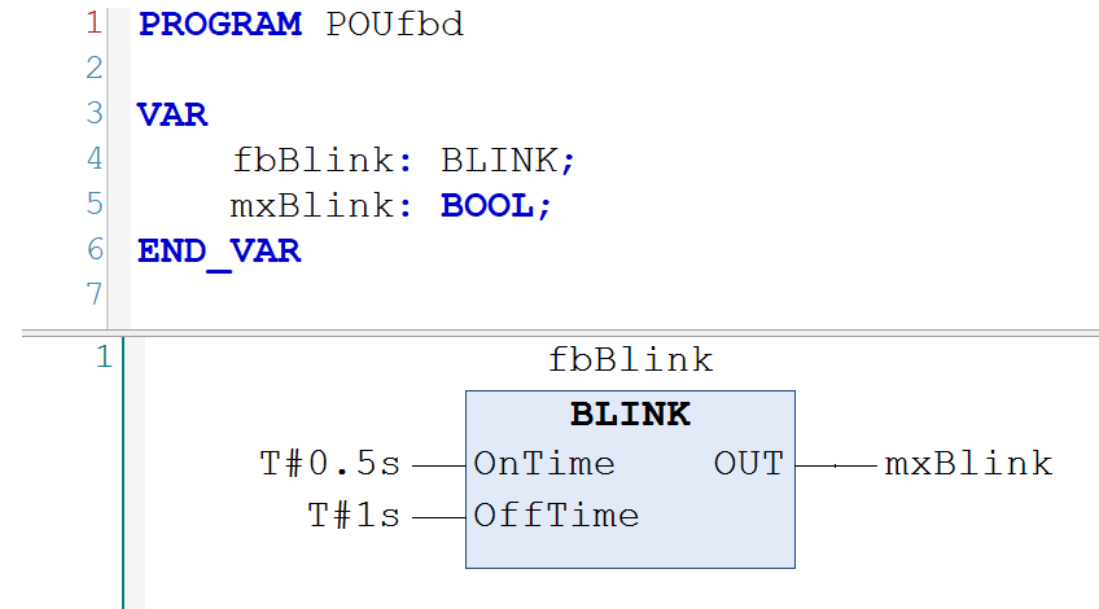


Blink Function Block from last class is used

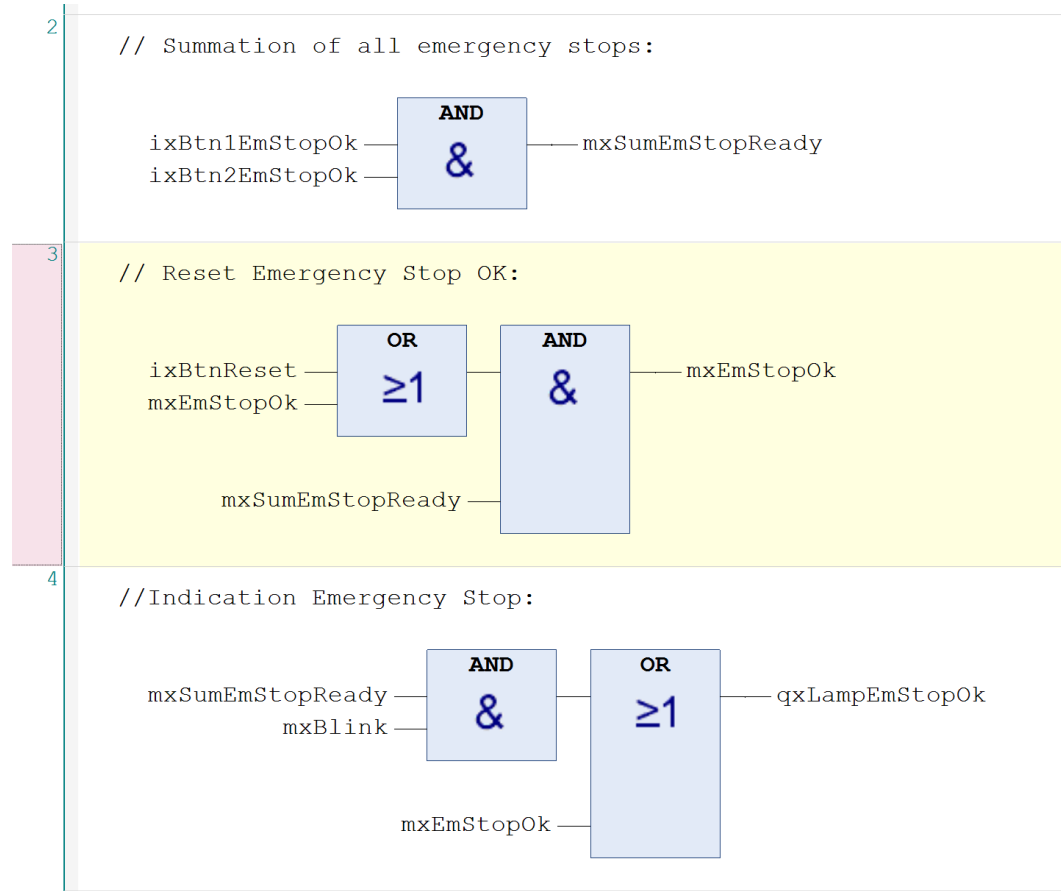


The screenshot shows the TwinCAT software interface. On the left is the 'Project Explorer' with a search bar and a list of projects and files. The selected project is 'TwinCAT Project4' (1 project). Under it, there is a folder 'POUs' which contains a file 'BLINK (FB)' that is currently selected. The main area on the right displays the ladder logic code for the 'BLINK' function block. The code is as follows:

```
1 FUNCTION_BLOCK BLINK
2 VAR_INPUT
3     OnTime : TIME;
4     OffTime : TIME;
5 END_VAR
6 VAR_OUTPUT
7     OUT : BOOL;
8 END_VAR
9 VAR
10     fbTon : TON;
11     fbToff : TON;
12 END_VAR
13
14 1 fbTon.PT := OnTime;
15 2 fbToff.PT := OffTime;
16
17 3 fbTon.IN := NOT fbToff.Q;
18 4 fbTon();
19
20 5 fbToff.IN := fbTon.Q;
21 6 fbToff();
22
23 7 Out := fbTon.Q;
```



Emergency stop

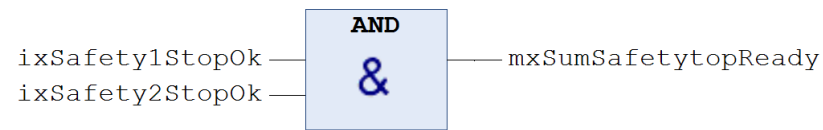


- All emergency stops are summated
- Reset of Em-Stop
- Indication, blink when ready to reset
- Steady light when reset

Safety stop

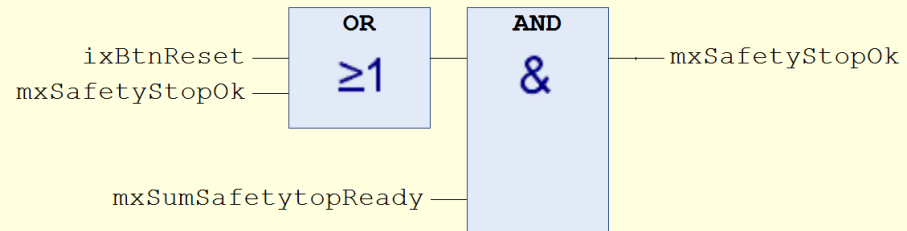
5

//Summation of all safetystops ready:



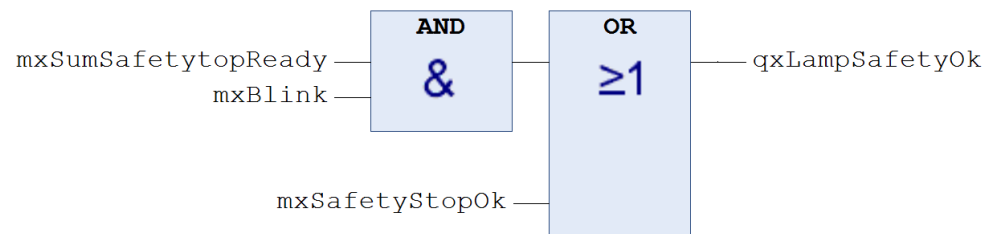
6

// Reset Safety Stop OK:



7

//Indication Safety Stop:



- All safety stops are summated

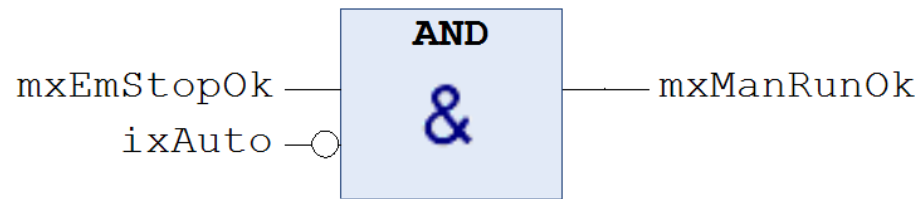
- Reset of Safety Stop

- Indication, blink when ready to reset
- Steady light when reset

Manual mode

8

```
// Run in manual mode OK:
```

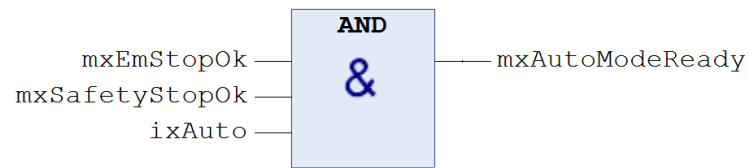


- Emergency stop OK and reset
- AND
- The mode selector NOT in auto (Manual mode)

Auto, home pos, cycle stop

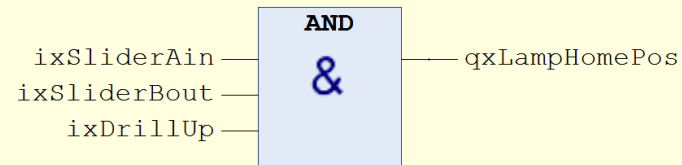
9

```
// Auto mode ready:
```



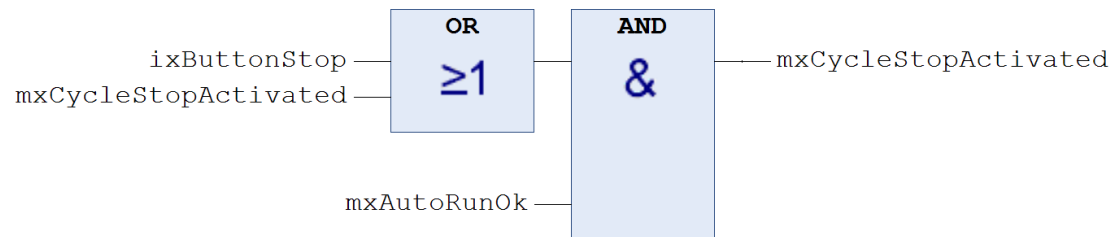
10

```
//Home position:
```



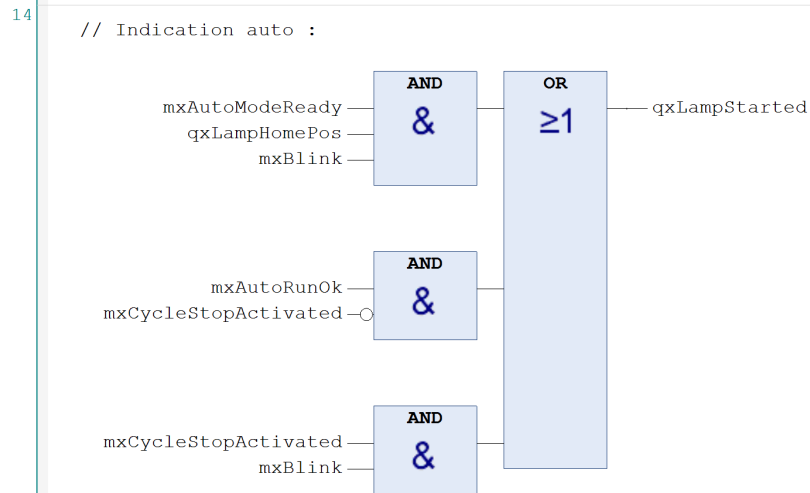
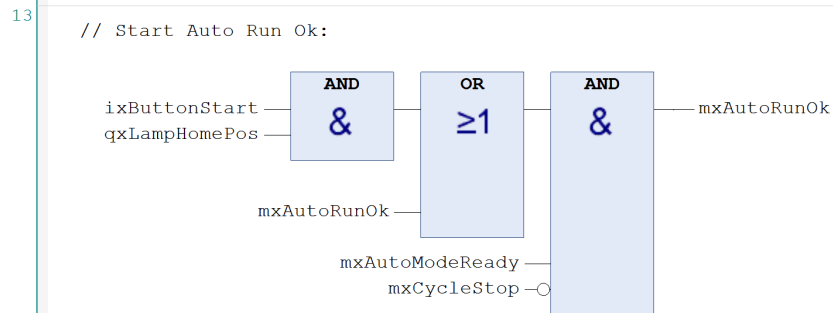
11

```
//Cycle stop activated:
```



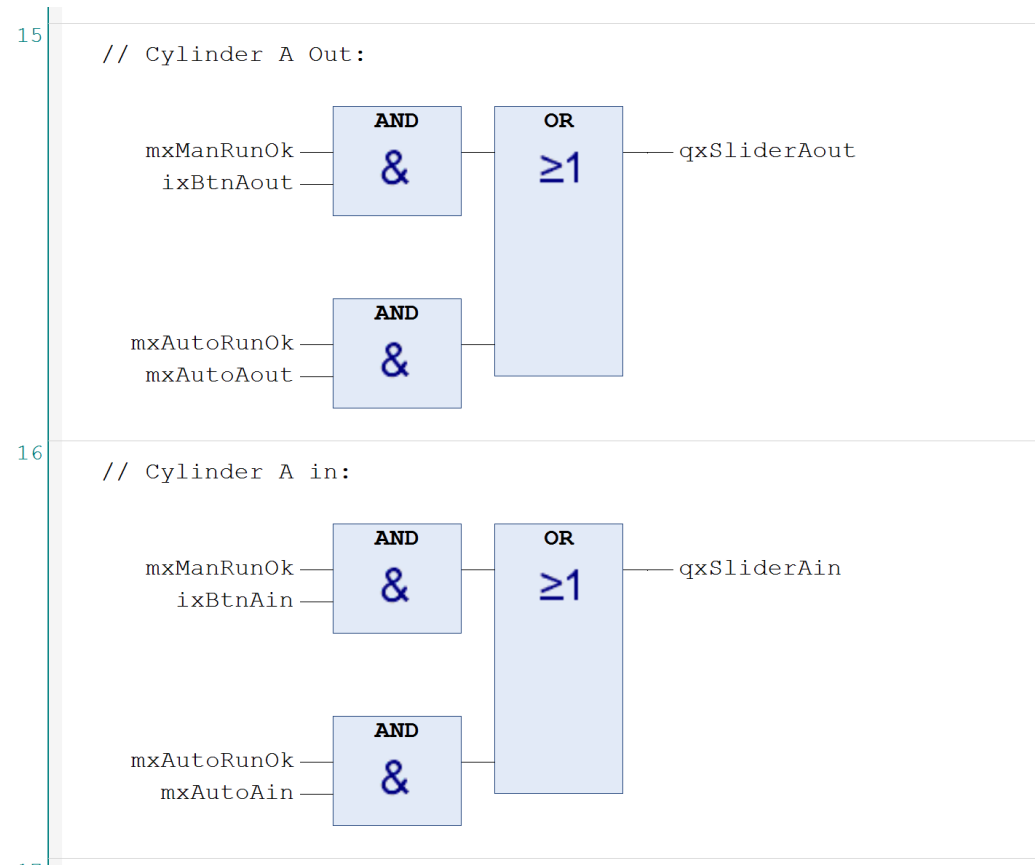
- Em-stop OK
- Safety stop OK
- Mode selector in Auto
- Home position
 - Safe to start auto
- Cycle stop
 - Stops auto at the end of the machine cycle

Autorun and indication



- Start auto only in the home position
 - Auto mode selected and ready
 - Stops on the cycle stop pulse in home
-
- Blinking when ready to start
 - Steady light when running auto
 - Blinking during cycle stop

All output collected at the end of the program

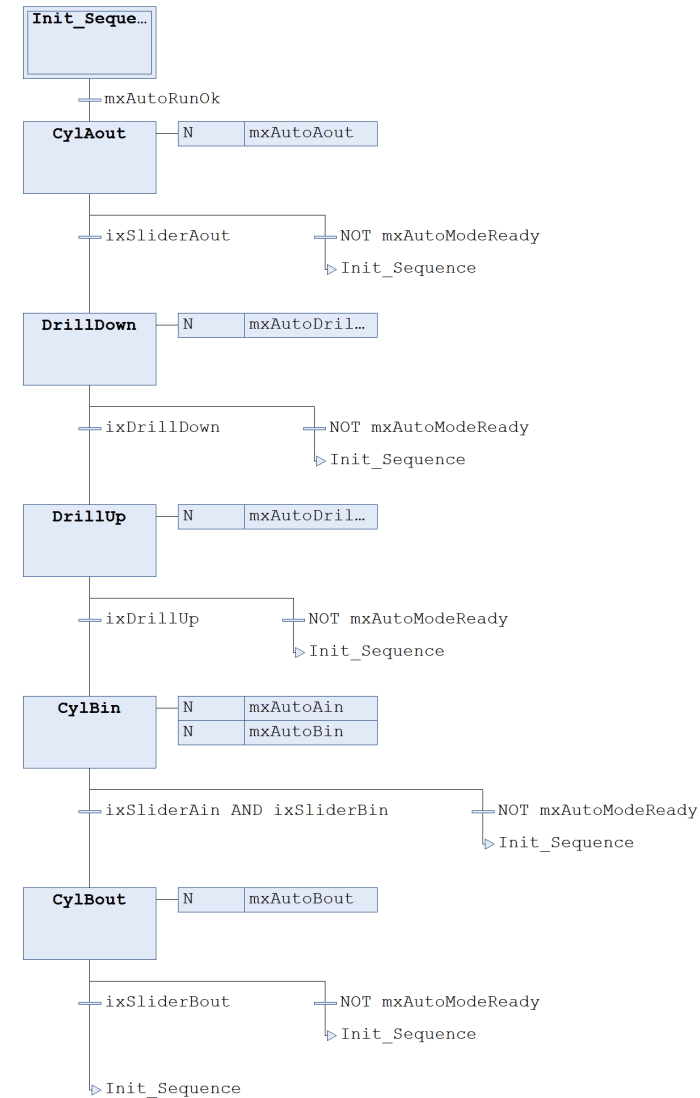


- Outputs for the sliders
- Manual mode
- or
- Auto mode

Outputs are often located in a separate POU

Auto sequence in SFC

- Single-line (Alternative) actions
- Double lines (Parallel) actions
- Arrows to jump out if not auto
 - Em-stop
 - Safety stop
 - Mode selector
- mxAutoRunOk
 - Transition from home
 - Stops in home



Sequence as a State machine in Structure Text

- Example of an ENUM in a CASE

 STATES (ENUM)

```
ControlFBD*      ControlFBD.Sequence  STATES  X  BLINK
1 {attribute 'qualified_only'}
2 {attribute 'strict'}
3 {attribute 'to_string'}
4 TYPE STATES :
5 (
6     init, Aout, drillDown, ....., .....
7 );
8 END_TYPE
9
```

```
miSeq: INT;
END_VAR

ControlFBD.sequenceST*  X
1
2 CASE miSeq OF
3
4     STATES.init:
5         IF mxAutoRunOk THEN
6             mxAutoAout := TRUE;
7             miSeq := STATES.Aout;
8         END_IF
9
10    STATES.Aout:
11        IF ixSliderAout THEN
12            mxAutoAout := FALSE;
13            mxAutoDrillout := TRUE;
14            miSeq := STATES.drillDown;
15        END_IF
16
17        //...
18 END_CASE
```